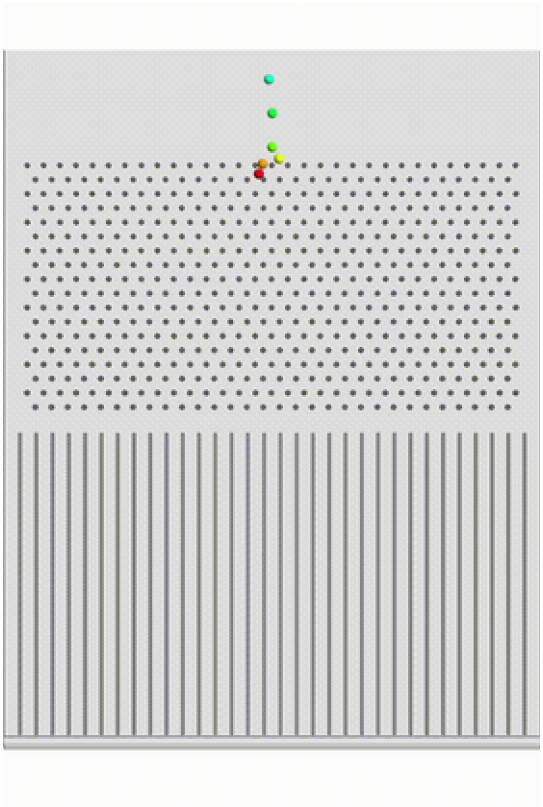
PhysicsModelLink in action: a Galton board, cascaded by Claude

A reply to Arnoud Buzing's PhysicsModelLink announcement — what happened when I pointed Claude at the paclet.

Marco Thiel, April 2026 — github.com/mthiel74/GaltonBoard

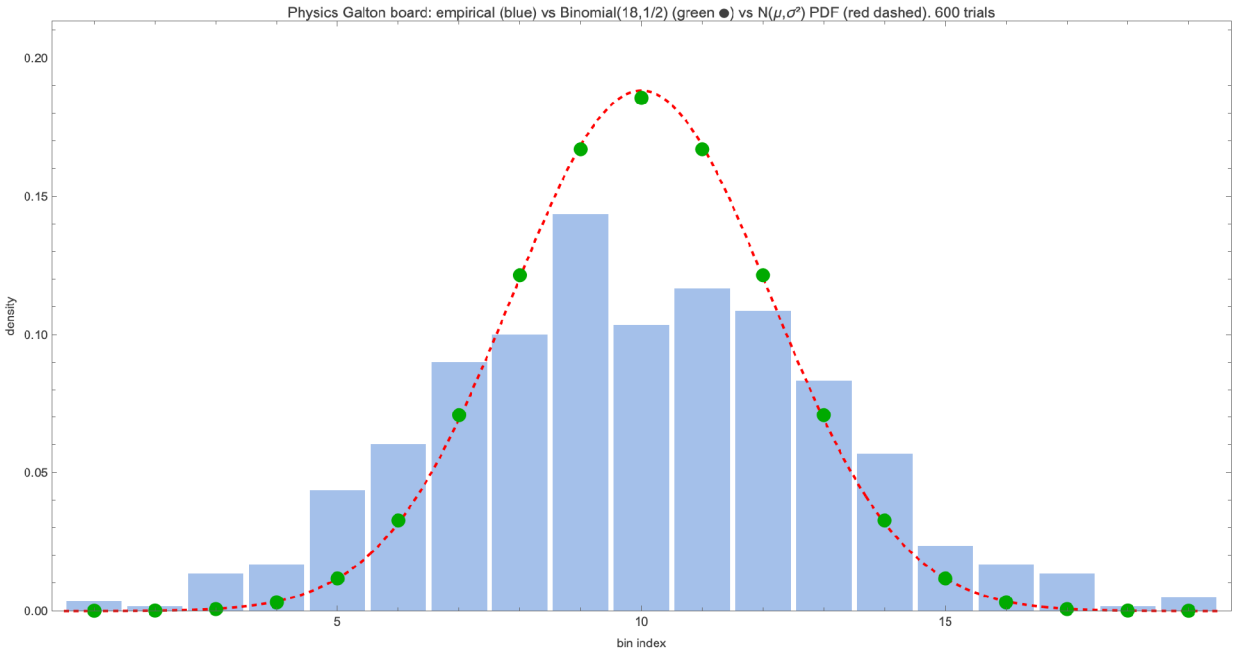
The result

Six hundred balls, dropped one after another into a narrow funnel above a 31×18 alternating-offset peg array. Each ball bounces off real `Cylinder` pegs (wrapped as `FixedBody`) under gravity, settles into one of 33 bins, and the pile in those bins forms the expected bell curve:



600 balls cascading through an 18-row Galton board. Real Rapier rigid-body collisions throughout; no virtual impulses.

The complementary statistical view — 600 independent single-ball simulations, final bin index compared against Binomial(18, 1/2) and the Normal approximation predicted by the central-limit theorem:



Empirical bin counts (blue bars) overlaid with the Binomial(18, 1/2) PMF (green dots) and the matching Normal PDF (red dashed).

How this came about

Arnoud's original post ends with an observation about LLMs lowering the cost of writing boilerplate wrapper code. I wanted to test a closely-related idea: *can you **point** a coding agent at a paclet and have it build a new project on top of that paclet?* I opened an empty GitHub folder and typed the following one-liner into Claude Code:

```
"Hi, this is an empty repo (please make an online one for this). I would like you to u
```

("Dalton" was a typo; it is a *Galton* board. I caught that a bit later and had Claude rename the repo, the package context and every reference in one go.) Claude followed the URL, ran `PacletInstall[ResourceObject["..."]]`, read the `PhysicsModelLink.wl` source to learn the `CreatePhysicsModel` / `DynamicBody` / `FixedBody` / `PhysicsModelEvolve` / `PhysicsModelVideo` API, and pushed a first working single-ball prototype within a few minutes. Between that prototype and the animation above were several iterations — not about writing new code, but about choosing geometry and parameters that actually produce the Binomial distribution:

- A "triangular" peg layout was eliminated because balls slid down the *outside* of the triangle and collected in the edge bins. A rectangular, alternating-offset lattice fixed that.
- The same-row peg-edge gap must be greater than the ball diameter (we ended up at 0.17 m vs 0.15 m), otherwise balls wedge.
- The outermost peg-to-wall gap has to satisfy the same constraint (fix: fewer pegs per row on the outside, walls further out).
- Rapier puts a body to sleep when its motion is very small; a ball that balances exactly on a peg top then sits there forever. A tiny per-step linear-velocity refresh (damping 0.99 every 4 physics steps) keeps the bodies awake and also approximates air drag.

- Per-trial peg-x jitter (~0.004 m) is the only non-deterministic ingredient; gravity, collisions and bounces are all real rigid-body physics.
- Pegs have to extend beyond the Binomial's 5σ range or residual chaos bunches balls in the extreme bins (we ended up with 31 columns over 25 bins of Binomial width).

Every one of these changes was *my* physical observation *("some pegs are horizontally closer than others — balls wedge there", "the balls slide along the outer edge of the triangle", "the outermost peg-to-wall gap is too small"*...) typed as a short sentence. Claude did the tuning and rebuilt the notebook. The total turn-around from observation to new render was a few minutes each time.

The physics layer

The whole board geometry lives in a tiny shared package, `GaltonBoard`\`, in scripts/board.wl. The parameters that ended up producing the bell curve are:

```
$PegRows    = 18;
$PegCols    = 31;                 (* 31 pegs in even rows, 30 in odd *)
$PegRadius  = 0.035;
$BallRadius = 0.075;              (* ball diameter 0.15 m *)
$PegDx      = 0.24;               (* same-row peg-edge gap = 0.17 m > ball dia *)
$PegDz      = 0.21;
$NBins      = 33;                 (* > Binomial 5-sigma range *)
$SlabHalfY  = 0.13;
```

Pegs are constructed with `FixedBody[{color, Cylinder[...]}, "Restitution" -> 0.5, "Friction" -> 0.0]` — zero friction so a ball that happens to balance on a peg top slides off rather than being grabbed by static friction.

```
                    to cover >5-sigma of the Binomial(18, 1/2) distribution
                    so extreme bins are statistically empty. *)

$NBins::usage = "$NBins = number of bins at the bottom. Odd so that bins are symm
$NBins = 33;

PegList::usage = "PegList[] returns the (pristine) rectangular peg grid.  PegList
PegList[] := PegList[0.0];
PegList[jitter_?NumericQ] := Module[{row, j, xj, x, pegs = {}, nPegs, firstX},
  Do[
    If[OddQ[row],
      (* odd row: one FEWER peg, offset so the row is symmetric about 0 *)
      nPegs  = $PegCols - 1;
      firstX = -(($PegCols - 2)/2.0) * $PegDx,
      (* even row *)
      nPegs  = $PegCols;
      firstX = -(($PegCols - 1)/2.0) * $PegDx
    ];
    Do[
      x  = firstX + j * $PegDx;
      xj = x + If[jitter > 0, RandomReal[{-jitter, jitter}], 0.];
      AppendTo[pegs,
        FixedBody[{GrayLevel[0.35],
          Cylinder[{{xj, -$SlabHalfY, -row*$PegDz},
                    {xj,  $SlabHalfY, -row*$PegDz}}, $PegRadius]},
          "Restitution" -> 0.5, "Friction" -> 0.0]],
```

Dropping the balls

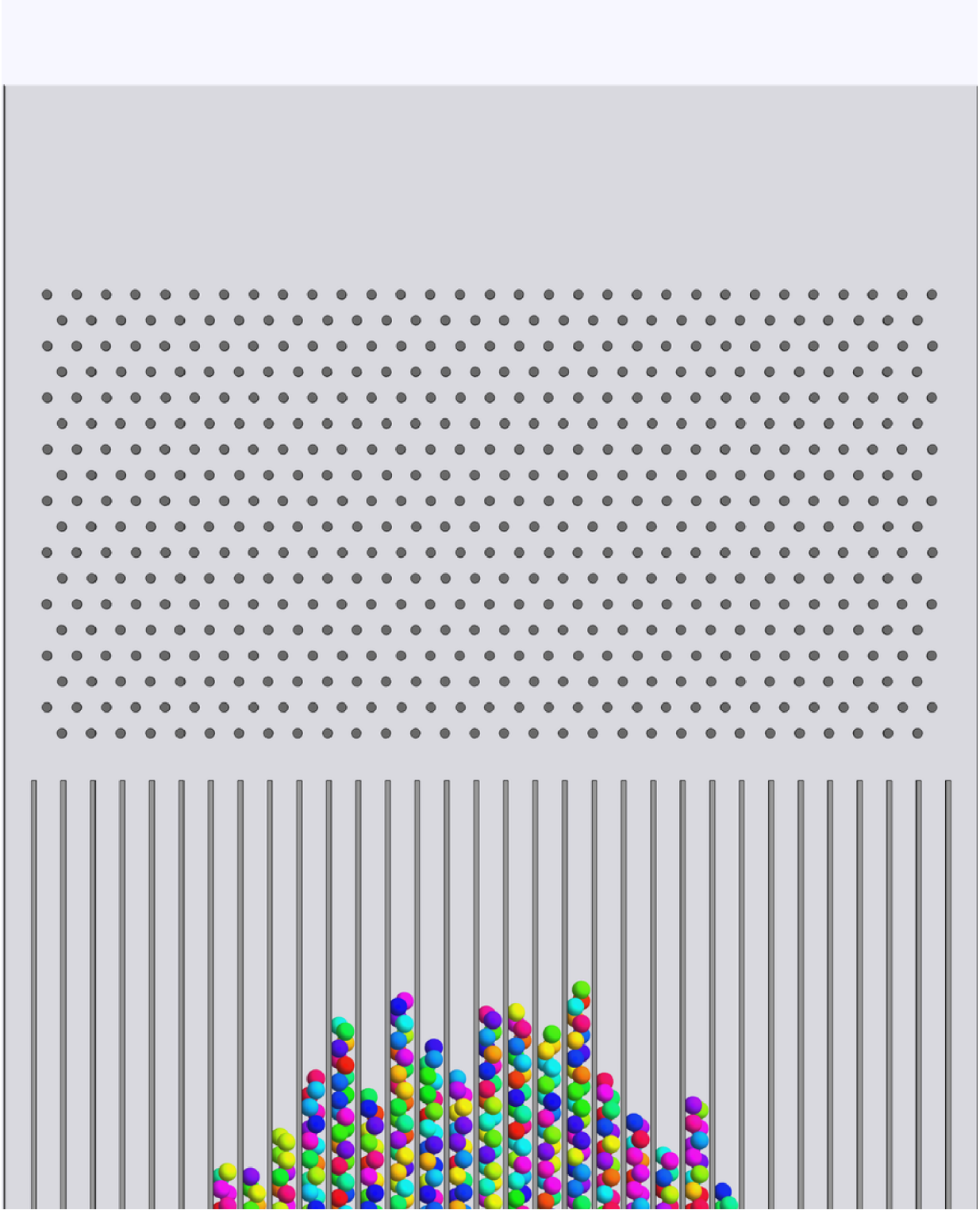
The animation script drops `nBalls = 600` balls — stacked vertically above the board with 0.5 m between them so each ball's arrival at the top peg is staggered by ~0.03 s. The only 'non-physical' ingredient is a gentle per-step velocity refresh that prevents Rapier's body-sleep mechanism from freezing balls on peg tops:

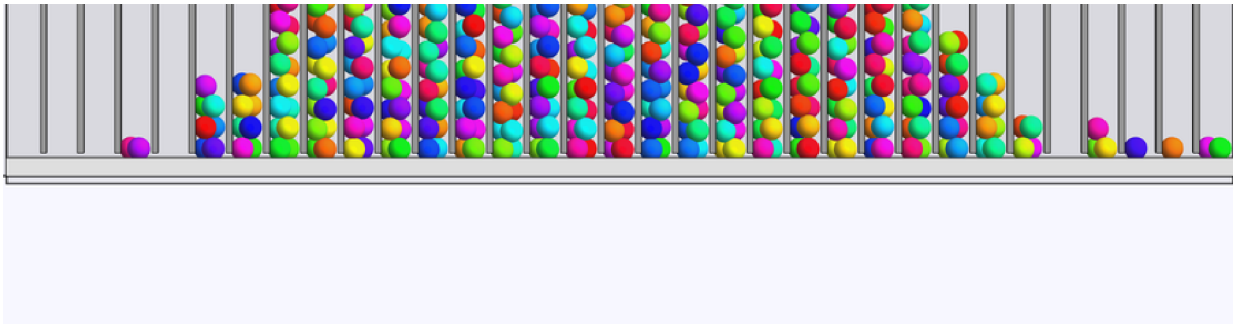
```
current  = model;
prevPoss = Table[current[[1]]["Bodies"][[k]]["Position"], {k, 1, nBalls}];
frames   = Reap[
  Do[
    current = PhysicsModelIterate[current, 1, dt];
    If[Mod[step, dampEvery] == 0,
      Do[Module[{curPos, vel},
          curPos = current[[1]]["Bodies"][[b]]["Position"];
          vel    = (curPos - prevPoss[[b]]) / (dampEvery * dt);
          RapierSetBodyVelocity[worldId, ballHandles[[b]], vel * damping];
          prevPoss[[b]] = curPos],
        {b, 1, nBalls}]];
    Sow[current],
    {step, 1, nFrames}]][[2, 1]];
```

Everything else — gravity (`{0, 0, -9.81}`), ball–peg collisions, ball–ball collisions, ball–wall collisions, energy loss through restitution — is native Rapier.

Independent-trial statistics

To decouple the distribution from any ball-ball interactions, a second script (scripts/03_histogram.wls) runs `N = 600` independent single-ball simulations and records the final bin. The loop is simply `CreatePhysicsModel → PhysicsModelIterate loop → BinIndex[finalX] → DestroyPhysicsModel`. That yields the blue bars in the overlay figure near the top of this post, tracking Binomial(18, 1/2) and the Normal PDF closely.





Final frame of the animation — the pile of 600 settled balls, one colour per ball, showing the bell curve built out of actual rigid bodies.

What I take from this

1. **A paclet is now a pointer.** The entire project started from a URL. Claude installed the paclet, read its source, used its high-level `CreatePhysicsModel / PhysicsModelEvolve` API, and turned out a working simulation without me writing a line of code. That is the same shape of interaction Arnoud describes for LLM-assisted LibraryLink work, scaled up one level: instead of pointing the LLM at a C header, I point it at a whole paclet.

2. **The hard part stayed hard — physically correctly parameterising a Galton board.** Running the simulation was instant. Getting it to actually produce the Binomial(18, 1/2) took six rounds of iteration, and all six were driven by *my* physical intuition ("look, balls are wedging between pegs there"). Claude could see the distribution counts and the image but could not, without being told, reason about the 5 mm of peg-edge clearance or the Rapier body-sleep heuristic. Collaborators, not replacements.

3. **Wolfram Community embedding works.** The animation above is an `AnimatedImage` constructed from frames of `output/02_board_animation.gif`. Community's HTML renderer serialises it as a plain looping `<img>` (the trick I learned from Marco's TrafficJAms post).

The whole project — seven evolutionary scripts, the Rapier parameterisation, 600 trials of single-ball statistics, the video, the GIF, the histogram — sits at github.com/mthiel74/GaltonBoard. Feedback and further parameter tweaks welcome.

Thanks Arnoud — the paclet is delightful to work with.